THINGS TO KNOW :

1. **Lab report must contain following sections: (order must be maintained)**
   a) **Title /Question**
   b) **Theory: The brief overview of the concept /techniques/algorithm used in the program**
   c) **Code: The complete code**
   d) **Output: Screenshot of the output**
2. **Output screen should be captured (use snipping tool), printed and attached in the report. Other contents must be handwritten.**
3. **Every Source code must include the printing statements to print following information after your main output:**

   Lab No.: Name: Roll No./Section :

4. **Contents should be written on single side of A4 sized paper.**
5. **The works must be submitted within specified deadline.**
6. **Cover page and contents page should be attached in the report appropriately.**

==========

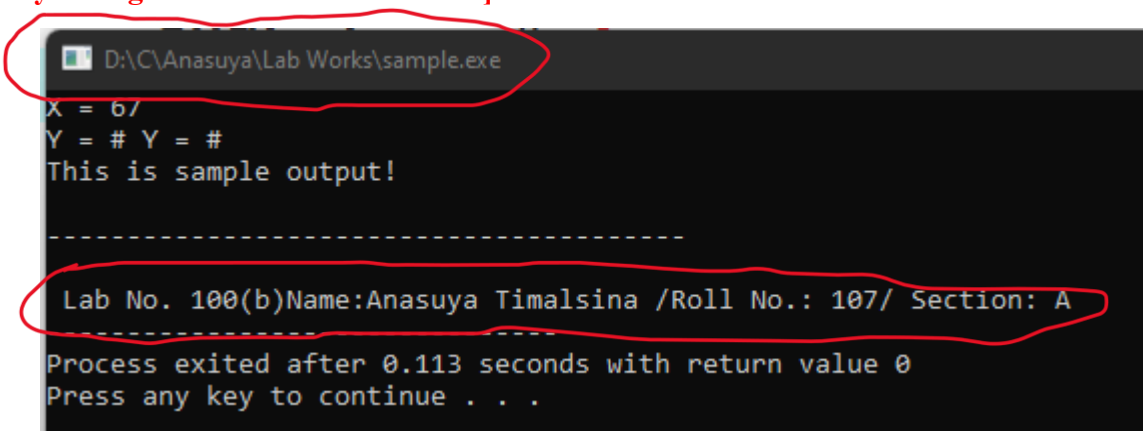
Contents Page Format (can be printed)

List of Lab Works

Lab No.	Title /Question	Submission Date	Signature	Remarks
1(a)	To print bio data	2078/08/22		
1(b)	To take two numbers and display their sum	2078/08/22		

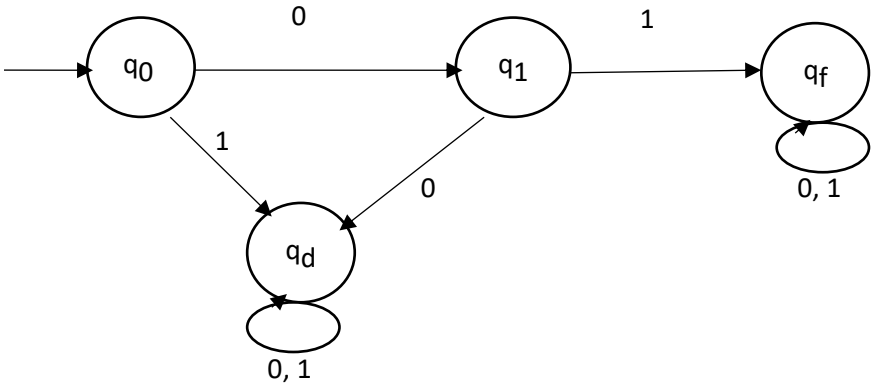
Sample output:

[show the top of window of output screen showing directory structure of your name and your signature line at the bottom]



Lab Works (Part -1)

1. **Write program to implement following DFA's over alphabet $\Sigma = \{0,1\}$.**
   a) The DFA that accepts all the strings that start with 01.



b) The DFA that accepts all the strings that end with 01.

Q = { q0, q1, qf}
start state = q0,
Final state = qf
Transition function, δ is defined as:

$$\delta(q0,0) = q1$$
$$\delta(q0,1) = q0$$
$$\delta(q1,0) = q1$$
$$\delta(q1,1) = qf$$
$$\delta(qf,0) = q1$$
$$\delta(qf,1) = q0$$

c) The DFA that accepts all the string that contains substring 001.

Q = { q0, q1, q2, qf}
start state = q0,
Final state = qf
Transition function, δ is defined as:

$$\delta(q0,0) = q1$$
$$\delta(q0,1) = q0$$
$$\delta(q1,0) = q2$$
$$\delta(q1,1) = q0$$
$$\delta(q2,0) = q2$$
$$\delta(q2,1) = qf$$
$$\delta(qf,0) = qf$$
$$\delta(qf,1) = qf$$

2. **Write a program to find prefixes, suffixes and substring from given string.**

3. **Write a C program to identify whether a given line is a comment or not**
   - Read the input string.
   - Check whether the string is starting '/' and check next character is '/'or '*'
   - If condition satisfies print comment.
   - Else not a comment.
4. **Write a program to validate C identifiers and keywords.**

   [ask to enter keyword and display whether it is valid or not, similarly ask to enter an identifier and display whether that is valid or not]

5. **Write a C program to simulate lexical analyzer for validating operators.**
6. **Write a C program for implementing symbol table.**
   - Start the program for performing insert, display, delete, search and modify option in symbol table
   - Define the structure of the symbol table.
   - Enter the choice for performing the operation in the symbol table.
   - If the entered choice is 1, search the symbol table for the symbol to inserted. if the symbol table is already present, it displays "Duplicate symbol". Else, insert the symbol table and the corresponding address in the symbol table.
   - If the entered choice is 2, the symbols present in the symbol table are displayed.
   - if the entered choice is 3, the symbol to be deleted is searched in the symbol table. If it is not found in the symbol table it displays "Label not found". Else the symbol is deleted.
   - If the entered choice is 4, search the symbol in the table
   - If the entered choice is 5, the symbol to be modified is searched in the symbol table. the label or address or both can be modified.
   - If the choice is 6, end the program

7. **Write a program to implement top-down parsing for a grammar**
8. **Write a C program to compute FIRST and FOLLOW of a grammar.**
9. **Write a program for implementing shift reduce parser using C.**
   - get the input expression and store it in the input buffer
   - read the data from the input buffer one at the time.
   - using stack and push and pop operation shift and reduce symbols with respect to production rules available.
   - Continue the process till symbol shift and production rule reduce reaches the start symbol.
   - Display the stack implementation table with corresponding stack actions with input symbols